

Worktree Isolation

Introduced: v2.1.49 (February 19, 2026) | First-class CLI: v2.1.50 | Latest: v2.1.76

What It Is

Worktree isolation creates independent copies of your git repository for parallel Claude Code sessions. Each worktree has its own files, branch, and working state — but shares git history and remotes. You can work on a feature in the main directory while a bug fix runs in a worktree, and a refactor runs in another. No file conflicts, no branch switching.

Why It Matters

Before worktrees, parallel work meant branch-switching gymnastics or multiple clones eating disk space. With worktrees: - Run 3 Claude sessions on different features simultaneously - Agent team members each get isolated workspaces - Experiment freely — worktrees auto-cleanup if you make no changes - No risk of one agent's edits breaking another's work

Quick Start (60 Seconds)

```
# Start Claude in a new worktree
claude --worktree
# Creates .claude/worktrees/<random-name>/ with a new branch

# Or name it
claude -w auth-feature
# Creates .claude/worktrees/auth-feature/ on branch worktree-auth-feature

# Inside Claude Code (interactive)
# Use EnterWorktree to create from within a session
```

CLI Flags

Flag	Short	Description
--worktree [name]	-w	Create worktree (optional name, random if omitted)
--tmux	—	Create tmux session for the worktree
worktree.sparsePaths—		Setting: sparse checkout for monorepos (v2.1.76)

```
# Named worktree
claude -w payment-refactor

# Worktree + tmux (see all in split panes)
claude -w payment-refactor --tmux

# Auto-generated name
claude -w
# Creates something like .claude/worktrees/bright-running-fox/
```

How Worktrees Work

```
your-repo/                                # Main working directory
├── src/
├── .claude/
│   └── worktrees/
│       ├── auth-feature/                # Worktree 1 (full repo copy)
│       │   ├── src/
│       │   └── ...
│       └── bugfix-123/                  # Worktree 2 (full repo copy)
│           ├── src/
│           └── ...
└── ...
```

Each worktree: - Has its own files (independent working tree) - Gets a new branch (worktree-<name>) - Shares .git history with the main repo - Shares remotes (can push/pull independently)

Cleanup Behavior

Scenario	What happens
No changes made	Worktree + branch auto-removed on exit
Uncommitted changes	Prompted: keep or remove
Commits exist	Prompted: keep or remove (default: keep)
Session interrupted	Worktree persists. Manual cleanup needed

Manual Cleanup

```
# List all worktrees
git worktree list

# Remove a specific worktree
git worktree remove .claude/worktrees/auth-feature

# Clean up stale worktree references
git worktree prune
```

Subagent Worktrees

Give each subagent its own isolated workspace:

In conversation

Run an Explore agent in an isolated worktree to analyze the authentication module without affecting my current work.

Or programmatically:

```
{
  "isolation": "worktree"
}
```

The subagent gets a temporary worktree that auto-cleans if no changes are made. If changes are made, the worktree path and branch are returned in the result.

Real-World Workflows

Workflow 1: Parallel Feature + Bug Fix

```
# Terminal 1: Main feature work
cd ~/my-project
claude
> implement user dashboard

# Terminal 2: Urgent bug fix (parallel)
cd ~/my-project
claude -w hotfix-login
> fix the login timeout bug, test it, and commit

# Terminal 3: Another feature (parallel)
cd ~/my-project
claude -w api-v2
> add v2 endpoints for the user API

# All three work independently. No conflicts.
```

Workflow 2: Agent Team with Worktrees

```
export CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS=1
claude

> Create a team where each member works in a worktree:
> - Backend agent: Add notifications API (worktree: notifications-api)
> - Frontend agent: Build notification UI (worktree: notification-ui)
> - Test agent: Write integration tests (worktree: notification-tests)
```

Workflow 3: Safe Experimentation

```
# Try a risky refactor in a worktree
claude -w experimental-refactor
> refactor the entire auth module to use passport.js
> [if it works, commit and merge]
> [if it doesn't, just exit - worktree auto-cleans]
```

Worktree Hooks

Automate worktree setup and teardown:

```
{
  "hooks": {
    "WorktreeCreate": [
      {
        "hooks": [
```

```

    {
      "type": "command",
      "command": "cd $WORKTREE_PATH && npm install"
    }
  ],
  "WorktreeRemove": [
    {
      "hooks": [
        {
          "type": "command",
          "command": "echo 'Worktree removed: $WORKTREE_PATH' >> ~/worktree.log"
        }
      ]
    }
  ]
}

```

Sparse Checkout for Monorepos (v2.1.76)

In large monorepos, worktree creation can be slow because it checks out the entire repo. The `worktree.sparsePaths` setting uses git sparse-checkout to only include the directories you need:

```

{
  "worktree": {
    "sparsePaths": [
      "packages/my-service/",
      "shared/",
      "configs/"
    ]
  }
}

```

This dramatically improves startup performance — instead of checking out 100K+ files, only the specified directories are materialized. Combined with the v2.1.76 fix for direct git ref reading, worktree creation is now fast even in very large codebases.

Non-Git VCS

For SVN, Perforce, or Mercurial, use `WorktreeCreate/WorktreeRemove` hooks to provide custom creation/cleanup logic.

How It Interacts With Other Features

Feature	Interaction
Agent Teams	Each teammate can work in its own worktree (no file conflicts)
Session Resume	Each worktree has separate session history. <code>-r</code> shows only that worktree's sessions
Hooks	<code>WorktreeCreate/WorktreeRemove</code> automate setup/cleanup

Feature	Interaction
CLAUDE.md	Each worktree gets a fresh copy of CLAUDE.md
MCP Servers	Inherited from parent repo config
Skills/Plugins	Custom agents and skills discovered in worktree directories
Background Tasks	Work correctly in worktrees (fixed v2.1.50)
tmux	--tmux creates split panes for worktree sessions

Gotchas & Tips

1. **Worktrees don't install dependencies** — run `npm install`, `pip install`, etc. in each worktree. Use `WorktreeCreate` hooks to automate this.
2. **Add to .gitignore** — add `.claude/worktrees/` to `.gitignore` to prevent untracked file noise.
3. **Branch name collisions** — `worktree-<name>` branch already exists? Use a different name. Check with `git branch -a`.
4. **Disk space** — each worktree is a full copy of your files (not git objects, those are shared). Monitor if you have many long-lived worktrees.
5. **IDE detection** — your IDE may not auto-detect worktree directories. Open the worktree folder explicitly if needed.
6. **Context isolation** — worktree sessions don't share conversation history with the main directory. Each is independent.
7. **Keep worktrees short-lived** — hours to days, not weeks. Commit work before cleanup.
8. **Manual cleanup** — `git worktree list` shows all worktrees. `git worktree remove <path>` cleans up.

Version History

Version	Date	Change
v2.0.x	Late 2025	Desktop app had worktree support
v2.1.49	Feb 19, 2026	--worktree / -w flag for CLI, isolation: worktree for subagents
v2.1.50	Feb 20, 2026	First-class CLI support, WorktreeCreate/WorktreeRemove hooks, background tasks in worktrees
v2.1.50	Feb 20, 2026	Custom agents and skills discovered in worktrees
v2.1.53	Feb 25, 2026	--worktree first-launch fix, stability improvements
v2.1.76	Mar 14, 2026	worktree.sparsePaths for sparse checkout, direct git ref reading for faster startup